

02-FRONTEND-PROJECTS

Frontend PROJECTS — l’application de production

React 19 · TypeScript 5.9 · Vite 7 · Tailwind CSS 4 · port **3000** Dossier : `frontend-projects/` C’est **l’application principale** : celle qui porte la page de connexion et le portail client.

Ce document contient aussi **le socle technique commun aux trois frontends** (sections 1 à 5). Les documents RH et Finance y renvoient.

SOMMAIRE

- 1. [Le socle technique commun aux 3 applications](#)
- 2. [L’arborescence des fichiers](#)
- 3. [Le démarrage : main.tsx et la pile de Providers](#)
- 4. [Le routage et les 4 types de garde](#)
- 5. [La couche API \(axios\)](#)
- 6. [Le layout et la navigation](#)
- 7. [Les modules métier de l’application Projets](#)
- 8. [Le portail client](#)
- 9. [L’assistant IA Media Plan](#)
- 10. [Thème, polices, accessibilité](#)
- 11. [Build et déploiement](#)

1. Le socle technique commun

1.1 Le monorepo npm workspaces

À la racine du projet, `package.json` :

```
{
  "name": "rh-application-monorepo",
  "private": true,
  "workspaces": ["packages/*", "frontend-projects", "frontend-rh", "frontend-finance"],
  "scripts": {
    "dev:projects": "npm run dev -w frontend-projects",
    "dev:rh": "npm run dev -w frontend-rh",
    "dev:finance": "npm run dev -w frontend-finance",
    "build:all": "npm run build:projects && npm run build:rh && npm run build:finance",
    "test:widget": "npm run test -w @antigone/ai-chat-widget"
  }
}
```

Ce que ça apporte concrètement : - Un seul `npm install` à la racine installe tout. - Le package `@antigone/ai-chat-widget` est **lié en local** : on le modifie, les trois applications voient la modification immédiatement, sans publication npm. - Les dépendances communes (React, Vite...) sont **dédupliquées** dans un seul `node_modules`.

1.2 La stack de chaque application

Couche	Outil	Version
UI	React	19.2
Langage	TypeScript	5.9
Build / dev server	Vite	7.3
CSS	Tailwind CSS	4.1 (plugin Vite natif)
Routage	React Router	7.13
HTTP	Axios	1.13

Graphiques	Recharts	3.7
Animations	Framer Motion	12.38
Icônes	react-icons	5.6

Spécifique à Projects et RH (absent de Finance) : - antd (Ant Design 5.24) — tableaux et formulaires complexes - three + @react-three/fiber + @react-three/drei — le **logo 3D** animé - moment — manipulation de dates dans les calendriers

1.3 Pourquoi Vite plutôt que Create React App ?

Critère	Vite	CRA (webpack)
Démarrage à froid	< 1 s (ESM natif)	20-60 s
Hot reload	instantané	plusieurs secondes
Build de production	Rollup, optimisé	webpack
Statut	actif	déprécié depuis 2023

1.4 Pourquoi Tailwind 4 ?

- Plugin Vite natif (@tailwindcss/vite) : plus de fichier postcss.config.js .
- **Mode sombre** géré nativement par la variante dark: — chaque classe a son équivalent sombre écrit sur place.
- Purge automatique : seules les classes réellement utilisées finissent dans le CSS.

2. L’arborescence des fichiers

```
frontend-projects/src/
├── main.tsx                ← point d'entrée : Providers imbriqués
├── App.tsx                 ← table de routage complète + gardes
├── index.css               ← Tailwind + variables CSS de la charte
├── vite-env.d.ts           ← typage des variables d'environnement
├── api/                    (29 fichiers) un service par domaine métier
│   ├── axios.ts            ← instance centrale + intercepteurs
│   ├── authService.ts
│   ├── projetService.ts
│   ├── tacheService.ts
│   ├── mediaPlanService.ts
│   ├── clientService.ts
│   └── ...
├── components/
│   ├── layout/
│   │   ├── MainLayout.tsx  ← coquille des pages internes
│   │   ├── ClientLayout.tsx ← coquille du portail client
│   │   ├── Sidebar.tsx     ← navigation filtrée par permissions
│   │   └── Header.tsx
│   ├── ui/                 (12) ← composants réutilisables
│   │   ├── Button · Modal · Badge · DataTable · StatCard
│   │   ├── ConfirmDialog · DatePickerInput
│   │   ├── Logo3D           ← logo Three.js
│   │   ├── AppSwitchButton  ← bascule entre les 3 applications
│   │   ├── LightPullThemeSwitcher
│   │   └── AnniversaireModal ← rappel des anniversaires
│   └── ai/
│       └── MediaPlanAiAssistant.tsx
├── context/
│   ├── AuthContext.tsx     ← utilisateur connecté (global)
│   └── NotificationContext.tsx ← notifications non lues (polling)
├── hooks/
│   ├── useTheme.ts         ← thème + police + taille de police
│   ├── useSidebar.ts       ← état ouvert/fermé de la barre latérale
│   ├── useConfirm.ts       ← dialogue de confirmation en promesse
│   └── useTachesObligatoires.ts
├── pages/                  (46 fichiers) ← une page = une route
├── types/index.ts          ← TOUS les types TypeScript partagés
└── utils/
    ├── authStorage.ts      ← stockage du jeton + SSO inter-applications
    └── notificationRules.ts
```

tous deux beaucoup de pages, car les deux applications partagent une base commune. **Ce qui distingue réellement les deux applications, c'est la table de routage de leur `App.tsx`** : seules les pages déclarées dans `App.tsx` sont accessibles. C'est là qu'il faut regarder pour savoir ce que chaque application expose vraiment.

3. Le démarrage : `main.tsx` et la pile de Providers

```
const basename = import.meta.env.BASE_URL || '/';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter basename={basename}>      {/* 1. routage */}
      <ThemeProvider>                        {/* 2. thème + police */}
        <AuthProvider>                      {/* 3. utilisateur connecté */}
          <SidebarProvider>                  {/* 4. état de la barre latérale */}
            <App />
          </SidebarProvider>
        </AuthProvider>
      </ThemeProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

L'ordre d'imbrication n'est pas arbitraire : - `BrowserRouter` en premier : les autres Providers peuvent avoir besoin de naviguer. - `ThemeProvider` avant `AuthProvider` : la page de connexion doit déjà être thématée. - `AuthProvider` avant `SidebarProvider` : la barre latérale se construit selon les permissions.

`basename={import.meta.env.BASE_URL}` : en développement `BASE_URL = '/'`, en production GitHub Pages `BASE_URL = '/Antigone_RH/projects/'`. React Router s'adapte sans changement de code.

`AuthContext` — les trois mécanismes de synchronisation

```
// 1. État initial lu depuis le stockage (ou relais inter-applications)
const [user, setUser] = useState(() => getAuthSnapshot()?.user ?? null);

// 2. Re-synchronisation périodique – détecte l'expiration de session
useEffect(() => {
  const interval = setInterval(() => { /* relit le snapshot */ }, 60_000);
  return () => clearInterval(interval);
}, []);

// 3. Écoute de l'événement `storage` – synchronise les onglets ouverts
useEffect(() => {
  window.addEventListener('storage', onStorage);
  return () => window.removeEventListener('storage', onStorage);
}, []);
```

Le point 3 mérite d'être montré : si l'utilisateur se déconnecte dans un onglet, l'événement `storage` du navigateur déclenche la déconnexion dans **tous les autres onglets** de la même origine, immédiatement.

4. Le routage et les 4 types de garde

`App.tsx` de `frontend-projects` définit **quatre composants de garde** :

4.1 `ProtectedRoute` — faut-il être connecté ?

```
const ProtectedRoute = ({ children }) => {
  const { isAuthenticated } = useAuth();
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  return <>{children}</>;
};
```

4.2 `PermissionRoute` — une permission précise

```
const PermissionRoute = ({ permission, children }) => {
  const { user } = useAuth();
  if (!user?.permissions?.includes(permission)) return <Navigate to="/dashboard" replace />;
  return <>{children}</>;
};
```

4.3 AnyPermissionRoute — au moins une parmi plusieurs

```
const AnyPermissionRoute = ({ permissions, children }) => {
  const hasAny = permissions.some(p => user?.permissions?.includes(p));
  if (!hasAny) return <Navigate to="/accueil" replace />;
  return <>{children}</>;
};
```

Utilisé pour le **Calendrier Projets**, accessible avec `VIEW_CALENDRIER_PROJETS` ou `VIEW_DEADLINES` ou `VIEW_REUNIONS`.

4.4 ClientRoute — réservé aux clients, page par page

```
const ClientRoute = ({ children, pageKey }) => {
  const { user, isAuthenticated } = useAuth();
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (!user?.isClient) return <Navigate to="/projets" replace />;
  if (pageKey) {
    const allowed = user?.clientPages ?? [];
    if (allowed.length > 0 && !allowed.includes(pageKey))
      return <Navigate to="/client/accueil" replace />;
  }
  return <>{children}</>;
};
```

Double vérification : **est-ce bien un client ?** puis **cette page lui est-elle ouverte ?** La liste vient de la colonne `Client.clientPages` en base.

4.5 SmartRedirect — la racine intelligente

```
const SmartRedirect = () => {
  const { user } = useAuth();
  return user?.isClient
    ? <Navigate to="/client/accueil" replace />
    : <Navigate to="/projets" replace />;
};
```

4.6 La table de routage complète

PUBLIC		
/login	/forgot-password	/reset-password
INTERNE (MainLayout + ProtectedRoute)		
/accueil	AccueilProjetsPage	libre
/projets	ProjetsPage	VIEW_PROJETS
/equipes	EquipesPage	VIEW_EQUIPES
/mes-taches	MesTachesPage	libre
/mon-profil	MonProfilPage	libre
/projets/:projetId/taches	ProjetTachesPage	VIEW_PROJETS
/admin/projets/:projetId/taches	AdminProjetTachesPage	VIEW_TOUS_PROJETS
/admin/calendrier-projets	CalendrierProjetsAdminPage	3 permissions possibles
/admin/clients	ClientsPage	VIEW_CLIENTS
/media-plan /media-plan/:clientId	MediaPlanPage	VIEW_MEDIA_PLAN
/admin/media-plans	TousLesMediaPlanPage	VIEW_TOUS_MEDIA_PLAN
/rapport-projet	RapportProjetPage	VIEW_PROJETS
/admin/dashboard-projets	AdminProjectDashboardPage	VIEW_TOUS_PROJETS
/admin/dashboard-projets/:projetId	AdminProjectDetailPage	VIEW_TOUS_PROJETS
/reactifs/intern /reactifs/extern	ReactifInternPage/ExternPage	VIEW_PROJETS
PORTAIL CLIENT (ClientLayout + ClientRoute)		
/client/accueil	ClientAccueilPage	—
/client/media-plans	ClientMediaPlansPage	pageKey MEDIA_PLANS
/client/projets	ClientProjectsDashboardPage	pageKey PROJETS
/client/fichiers	ClientDriveFilesPage	pageKey FICHIERS

5. La couche API (axios)

api/axios.ts — l'instance centrale

```
// Détection automatique de l'URL du backend :
// 1. VITE_API_URL si définie
// 2. localhost → http://localhost:8080
// 3. sinon → https://antigone-rh.onrender.com
export const API_BASE = envUrl
  ? envUrl
  : (hostname === 'localhost' || hostname === '127.0.0.1')
    ? 'http://localhost:8080'
    : 'https://antigone-rh.onrender.com';
```

```
const api = axios.create({
  baseURL: `${API_BASE}/api`,
  withCredentials: true,
  headers: { 'Content-Type': 'application/json' },
});
```

Les deux intercepteurs

```
// ENTRÉE : ajoute le jeton à chaque requête
api.interceptors.request.use((config) => {
  const token = getAccessToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// SORTIE : purge et redirige sur 401
api.interceptors.response.use(
  (r) => r,
  (error) => {
    if (error.response?.status === 401) {
      clearAuthSnapshot();
      if (window.location.pathname !== '/login') window.location.assign('/login');
    }
    return Promise.reject(error);
  }
);
```

Le bénéfice : aucune page du projet ne manipule le jeton ni ne gère l'expiration de session. C'est centralisé en 15 lignes.

Un service par domaine (29 fichiers)

```
// api/projetService.ts - exemple de structure
export const projetService = {
  getAll: () => api.get<ApiResponse<ProjetDTO[]>>(`/projets`),
  getById: (id) => api.get(`/projets/${id}`),
  create: (data) => api.post(`/projets`, data),
  update: (id, d) => api.put(`/projets/${id}`, d),
  delete: (id) => api.delete(`/projets/${id}`),
};
```

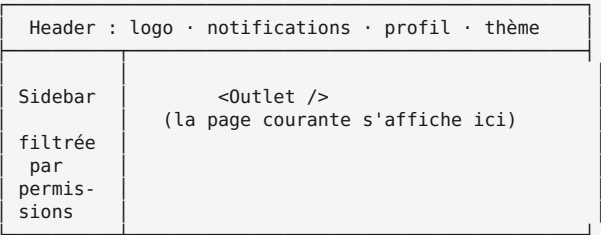
Liste complète : authService, employeeService, compteService, roleService, projetService, tacheService, tacheObligatoireService, equipeService, clientService, mediaPlanService, mediaPlanAssignmentService, mediaPlanCommentService, reactifService, demandeService, demandePapierService, validationService, calendrierService, calendrierProjetService, reunionService, referentielService, competenceService, documentEmployeeService, notificationService, projetAnalyseService, adminDashboardProjectService, agentDashboardService, agentHistoriqueService, rapportInactiviteService.

types/index.ts — le contrat avec le backend

Un seul fichier contient **toutes** les interfaces TypeScript miroir des DTOs Java. Conséquence pratique : si le backend renomme un champ, **le compilateur TypeScript signale toutes les pages impactées** au `npm run build`.

6. Le layout et la navigation

MainLayout.tsx



```
[AppSwitchButton] ← flottant, en bas à droite
```

Sidebar.tsx — navigation à deux niveaux

Structure de données :

```
type RailItem = { key, label, icon, path, permission?, prefixes[] };
type PanelItem = { key, label, path, icon, permission?, children?: {label, path}[] };
```

Le rail vertical porte les grandes familles (Accueil, Projets, Calendrier, Media Plan, Rapports, Clients) ; un panneau latéral s’ouvre au clic avec le détail.

Les entrées sont filtrées par permission :

```
const normalizePermission = (p: string) => p.trim().toUpperCase();
// puis, pour chaque item : user.permissions.includes(item.permission)
```

Fonctionnalité remarquable : le menu Media Plan et le menu Projets se **construisent dynamiquement** à partir de la liste des clients accessibles :

```
children.push({ label: client.nom, path: `/media-plan/${client.id}` });
```

Chaque marque devient une entrée de menu.

Le menu Calendrier ajoute ses onglets selon les droits :

```
if (has('VIEW_DEADLINES')) children.push({ label: 'Deadlines', path: '...?tab=deadlines' });
if (has('VIEW_REUNIONS')) children.push({ label: 'Réunions', path: '...?tab=reunions' });
if (has(...)) children.push({ label: 'Tournage', path: '...?tab=tournage' });
```

AppSwitchButton.tsx — la bascule inter-applications

```
const APPS = [
  { kind: 'projects', label: 'Projets', emoji: '🏢', envKey: 'VITE_PROJECTS_APP_URL' },
  { kind: 'rh', label: 'RH', emoji: '👥', envKey: 'VITE_RH_APP_URL' },
  { kind: 'finance', label: 'Finance', emoji: '💰', envKey: 'VITE_FINANCE_APP_URL' },
];

// L'espace Finance n'est proposé qu'aux comptes qui en ont la permission
const canViewFinance = user?.permissions?.includes('VIEW_FINANCE') || user?.roles?.includes('ADMIN');

const switchTo = (target) => {
  relayAuthSnapshotForSwitch(); // ← écrit le jeton dans window.name
  window.location.href = targetUrl;
};
```

L’application sait qui elle est grâce à `VITE_APP_KIND` (`projects` | `rh` | `finance`). Le bouton est repliable (`collapsed`) pour ne pas gêner.

7. Les modules métier

7.1 MediaPlanPage.tsx — 1 940 lignes, la page la plus importante du projet

C’est le cœur du métier de l’agence. Elle gère :

Fonction	Détail
Grille éditoriale mensuelle	Une ligne = une publication (date, heure, plateforme, format, type, titre, texte sur visuel, inspiration)
Sélection du client	Via l’URL <code>/media-plan/:clientId</code> ou un sélecteur
Navigation mensuelle	Mois précédent / suivant
Édition en ligne	Modification directe dans la grille
Statut de publication	<code>EtatPublication</code> (pas encore / publié...)
Rectifs et remarques	Champs texte de correction
Lien Drive	Dossier du mois, généré automatiquement
Bloc Tournage	Bascule <code>isShooting</code> → crée un projet + des tâches côté backend
Workflow de partage	Bouton « Partager » → statut envoyé au client
Commentaires	Fil de discussion par ligne, y compris avec le client
Assistant IA	Bouton dans l’en-tête → génération du media plan du mois

7.2 ProjetsPage.tsx (1 369 lignes)

Liste, création et édition des projets : statut, dates, type (déterminé/indéterminé), plusieurs chefs de projet, description, client rattaché.

7.3 MesTachesPage.tsx (892 lignes)

Vue personnelle des tâches assignées, filtrable par client (/mes-taches?clientId=...). Changement de statut, indicateur d’urgence, échéances, lien Drive.

7.4 AdminProjectDashboardPage / AdminProjectDetailPage

Tableau de bord direction (Recharts) : avancement par projet, charge par employé, tâches en retard, alertes. Le détail affiche un projet et toutes ses tâches.

7.5 RapportProjetPage.tsx — le rapport « Cycle de Vie »

Consomme ProjetAnalyseService : mesure les délais entre l’affectation d’une tâche et son démarrage réel, signale les dépassements (24 h manager, 8 h membre). **C’est un outil d’analyse de la productivité, pas un simple listing.**

7.6 ReactifInternPage / ReactifExternPage — les « Rectifs »

Suivi des demandes de correction. Chaque rectif indique **combien de fois** un livrable a déjà été renvoyé (nombreFois) — indicateur de qualité direct.

7.7 CalendrierProjetsAdminPage.tsx (934 lignes)

Calendrier unifié à onglets : **Deadlines, Réunions, Tournage**. Chaque onglet est gardé par sa propre permission.

7.8 ClientsPage.tsx (928 lignes)

Fiche marque complète : identité, activité, positionnement, objectifs, contacts, matricule fiscal, RNE, cycle de facturation, logo — **et les identifiants du portail client** (loginClient , passwordClient , clientPages).

Les champs identite , activite , positionnement , objectifs ne sont pas décoratifs : **ils alimentent le RAG de l’IA** pour la génération de media plan. Plus la fiche est remplie, meilleure est la génération.

7.9 EquipesPage.tsx

Composition des équipes (relation N:N employés ↔ équipes) et affectation des tâches obligatoires.

8. Le portail client

Un espace séparé, avec son propre layout (ClientLayout) et sa propre logique d’authentification (POST /api/auth/client-login).

Page	pageKey	Contenu
ClientAccueilPage	— (toujours ouvert)	Page d’accueil de la marque
ClientMediaPlansPage (773 l.)	MEDIA_PLANS	Le client consulte son calendrier éditorial, commente et valide
ClientProjectsDashboardPage	PROJETS	Avancement de ses projets
ClientDriveFilesPage	FICHIERS	Ses fichiers Google Drive (lecture seule)

Trois niveaux de sécurité empilés

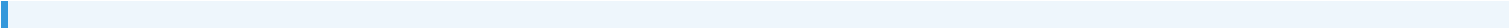
1. Authentification

→ le client a-t-il un jeton valide ?
2. Type de principal

→ user.isClient === true ?
3. Page autorisée

→ clientPages contient-il ce pageKey ?

Et côté backend, un **quatrième** niveau : ClientViewerDriveService utilise un compte de service Google **en lecture seule** (DRIVE_READONLY). Même compromis, un client ne peut rien modifier sur le Drive de l’agence.



Excellent argument de soutenance : « Le portail client est le point d'entrée le plus exposé de l'application, puisque des personnes extérieures à l'entreprise s'y connectent. J'ai donc empilé quatre contrôles indépendants, dont un au niveau de l'intégration Google. »

9. L'assistant IA Media Plan

Un montage différent des autres applications

```
/**
 * Montage du widget de chat IA, cantonné à la page Media Plan.
 *
 * Contrairement à RH/Finance où l'assistant est global, il n'est pertinent ici
 * que dans le contexte d'un media plan précis (capacité MEDIA_PLAN) : il vit
 * donc dans MediaPlanPage, pas dans le layout applicatif.
 */
<AiChatWidget
  apiUrl={API_BASE}
  getAuthToken={getAccessToken} // ← même source que l'intercepteur axios
  onUnauthorized={handleUnauthorized}
  capability="MEDIA_PLAN"
  isAdmin={isAdmin}
  onShareMediaPlan={onShareMediaPlan} // ← le résultat retourne DANS la page
  launcherLabel="Assistant Media Plan"
  hideLauncher // le déclencheur est dans l'en-tête de page
  theme={{ mode: theme, light: {...}, dark: {...} }}
/>
```

Le point d'architecture à souligner

« Le widget ne gère aucune authentification : il reçoit le jeton via `getAuthToken`, la même source que l'intercepteur axios de l'application. »

Une seule chaîne d'authentification existe par application. Le widget ne lit jamais `localStorage`, ni cookie, ni `window.name` par lui-même. C'est une décision de conception explicite : un composant réutilisable ne doit pas créer sa propre notion de session.

Le callback `onShareMediaPlan`

C'est le pont IA → application. Quand l'IA produit une proposition de media plan, le widget émet un `structured_result`. La page le reçoit via ce callback, affiche les publications proposées **sous forme de cartes**, et l'utilisateur peut alors les valider et les enregistrer. **L'IA ne persiste jamais rien elle-même.**

10. Thème, polices, accessibilité

`hooks/useTheme.ts`

Trois préférences persistées : **thème** (clair/sombre), **police** (Inter / Poppins), **taille de police** (13 / 14 / 15 / 16 px).

```
// Cookies partagés entre tous les ports du même domaine
// (contrairement à localStorage)
export const setSharedCookie = (name, value) => {
  document.cookie = `${name}=${value};expires=...;path=/;SameSite=Lax`;
};
```

Le détail malin : en développement, les trois applications tournent sur `localhost:3000`, `:3001`, `:3002`. Le `localStorage` est **isolé par port**, mais les **cookies sont partagés** par domaine. En stockant le thème dans un cookie, l'utilisateur qui passe de RH à Finance retrouve son mode sombre.

Ordre de priorité au démarrage :

```
cookie partagé > localStorage > prefers-color-scheme du système
```

`Logo3D.tsx`

Logo en 3D via Three.js / React Three Fiber (`cccc.glb` à la racine du projet). Présent dans Projects et RH, **volontairement absent de Finance** : une application comptable n’a pas besoin de charger 500 Ko de moteur 3D.

11. Build et déploiement

`vite.config.ts`

```
export default defineConfig({
  base: process.env.GITHUB_ACTIONS ? '/Antigone_RH/projects/' : '/',
  plugins: [react(), tailwindcss()],
  server: {
    port: 3000,
    proxy: { '/api': { target: 'http://localhost:8080', changeOrigin: true } },
  },
});
```

Deux choses à retenir : - `base` **conditionnel** : en local `/`, sur GitHub Pages `/Antigone_RH/projects/`. - **Proxy de développement** : les appels `/api` sont relayés vers `localhost:8080` → **pas de problème de CORS en développement**.

Le script de build

```
"build": "tsc -b && vite build && node -e \"require('node:fs').copyFileSync('dist/index.html','dist/404.html')\""
```

Trois étapes : 1. `tsc -b` — **vérification des types** : le build échoue si un type est incorrect. 2. `vite build` — bundle de production. 3. **Copie de `index.html` en `404.html`** — l’astuce GitHub Pages : quand l’utilisateur recharge `/projects/media-plan`, GitHub ne trouve pas de fichier et sert `404.html`, qui contient l’application et laisse React Router résoudre la route.

Variables d’environnement (`.env.production`)

```
VITE_API_URL=https://antigone-rh.onrender.com
VITE_APP_KIND=projects
VITE_PROJECTS_APP_URL=https://.../Antigone_RH/projects
VITE_RH_APP_URL=https://.../Antigone_RH/rh
VITE_FINANCE_APP_URL=https://.../Antigone_RH/finance
```